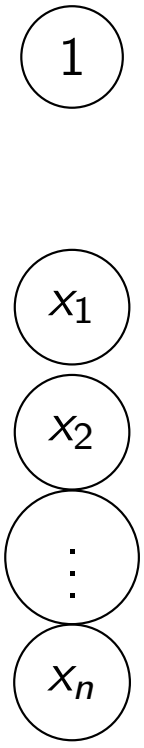
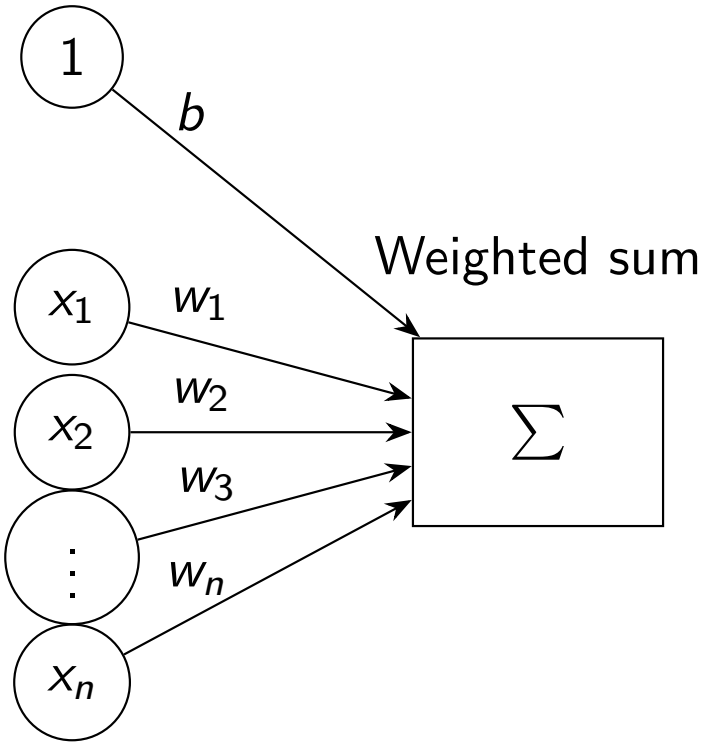
Perceptron: Theory, Learning, and Boolean Applications

Moin Mostakim

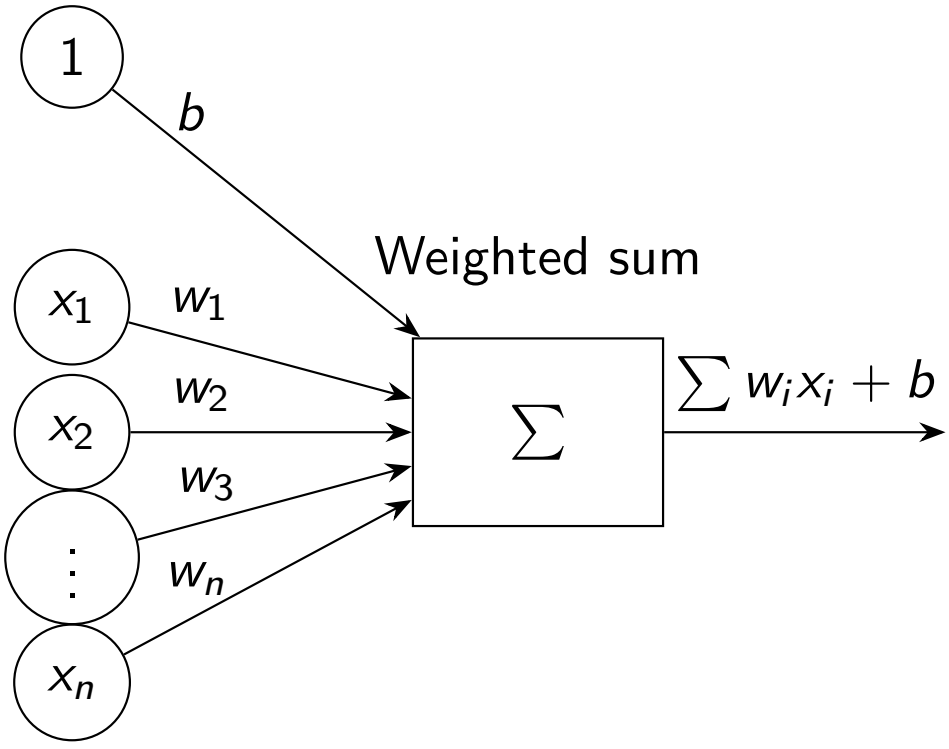
Perceptron — Basic Model



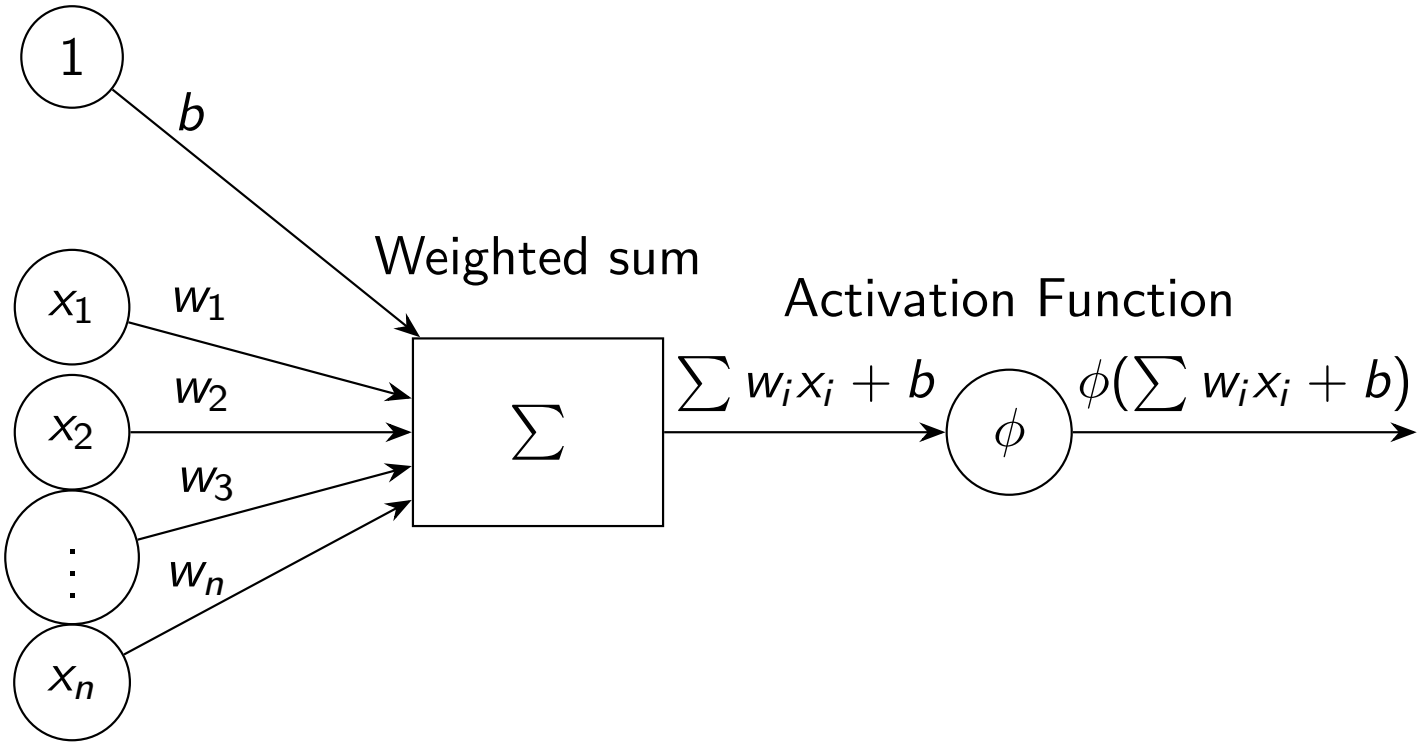
Perceptron — Basic Model



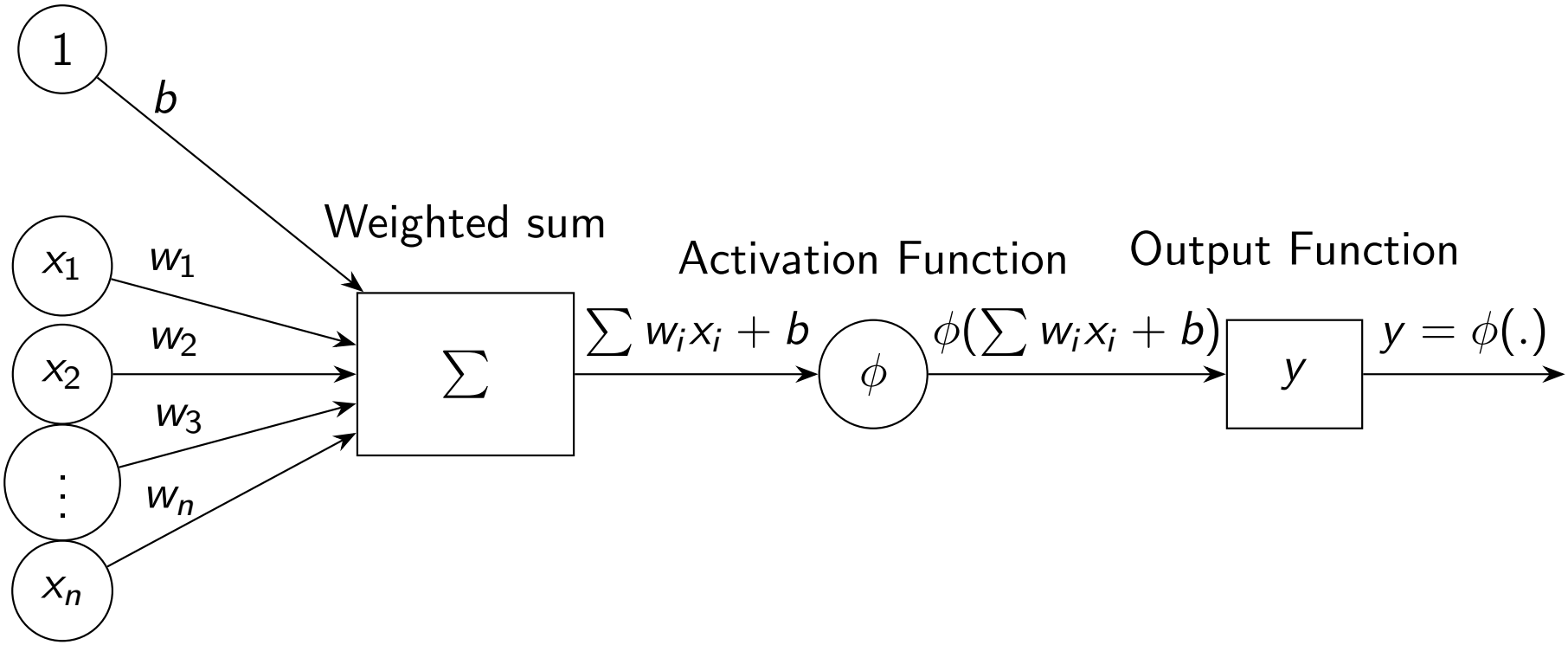
Perceptron — Basic Model



Perceptron — Basic Model



Perceptron — Basic Model



Core Equation

Core equations

$$z = \mathbf{w} \cdot \mathbf{x} + b$$

$$y = \phi(z) = \begin{cases} 1 & z > 0 \\ 0 & z \leq 0 \end{cases}$$

$$\Delta w_i = \eta(t - y)x_i, \quad \Delta b = \eta(t - y)$$

Components

$z =$ **Weighted Sum**

$\phi(.) =$ **Activation Function**

$\Delta w_i =$ **Weight differences**

$\Delta b =$ **Bias differences**

$\eta =$ **Learning rate**

Error Calculation Objective

Error and loss functions

- Classification error: $e = t - y$
- Perceptron criterion: $L_P(\mathbf{w}) = -\sum_{\text{mis}} t(\mathbf{w} \cdot \mathbf{x} + b)$
- Alternative (for analysis): $E = \frac{1}{2}\sum(t - y)^2$

Convergence Theorem

If data are linearly separable, perceptron converges in finite updates.
Bound on number of mistakes:

$$M \leq \left(\frac{R}{\gamma}\right)^2,$$

where R = max norm of inputs, γ = margin.

Applications and Limitations

Applications

- Binary classification (spam, signal thresholding)
- Basis for MLPs and feature detectors
- Used in linear classifiers and simple pattern recognition

Limitations

- Only linearly separable problems solvable
- Cannot model XOR/parity
- Discrete output, no probability interpretation
- Sensitive to feature scaling and noise

Boolean Functionality — Mathematical Depth

Linear separability of Boolean functions

A Boolean function $f : \{0,1\}^n \to \{0,1\}$ is linearly separable if $\exists \mathbf{w}, b : \; f(x) = 1_{\{\mathbf{w}\cdot x + b > 0\}}$.

Examples (2-input)

- AND: $w = (1,1), b = -1.5$ separable
- OR: $w = (1,1), b = -0.5$ separable
- XOR: not separable (no single hyperplane works)

Counting

Total Boolean functions: 2^{2^n}

Linearly separable counts: $n = 1 : 4,\ n = 2 : 14,\ n = 3 : 104$ (of 256 total)

Feature Maps for Nonlinear Problems

Mapping to higher dimension

Example (to make XOR linear):

$$\phi(x_1, x_2) = (x_1, x_2, x_1 x_2)$$

Then $x_1 \oplus x_2 = x_1 + x_2 - 2x_1 x_2$ (linear in ϕ).

Perceptron Learning Algorithm

Algorithm (Pseudo-code)

1. Initialize weights w_i and bias b to small random values.
2. For each training sample $(\mathbf{x}, t)$:
   - Compute $y = \phi(\mathbf{w} \cdot \mathbf{x} + b)$
   - Update: $w_i \leftarrow w_i + \eta(t - y)x_i, \quad b \leftarrow b + \eta(t - y)$
3. Repeat for all samples until convergence or max epochs.

Remarks

Learning rate η controls the step size. If data are linearly separable, weights converge in finite iterations (Perceptron Convergence Theorem).

Worked Example: Learning AND Gate

Training Data

x_1	x_2	t
0	0	0
0	1	0
1	0	0
1	1	1

Initialization

$w_1 = 0,\ w_2 = 0,\ b = 0,$
$\eta = 1$

Iterations

- Sample (0,0): $y = 0,\ t = 0 \rightarrow$ no change
- Sample (0,1): $y = 0,\ t = 0 \rightarrow$ no change
- Sample (1,0): $y = 0,\ t = 0 \rightarrow$ no change
- Sample (1,1): $y = 0,\ t = 1 \rightarrow$ $\Delta w_1 = 1,\ \Delta w_2 = 1,\ \Delta b = 1$

Final Model

$w_1 = 1,\ w_2 = 1,\ b = -1.5$ (adjust bias for correct threshold)
Decision: $y = 1$ if $x_1 + x_2 > 1.5$, else 0.

Threshold and Bias in Perceptron

Bias (b) Definition

The bias allows the perceptron to **shift the decision boundary** away from the origin.

$$y = \phi(\mathbf{w} \cdot \mathbf{x} + b)$$

- If $b = 0$, the hyperplane passes through the origin.
- Positive b shifts the hyperplane **left/up**.
- Negative b shifts the hyperplane **right/down**.

Threshold (θ) Concept

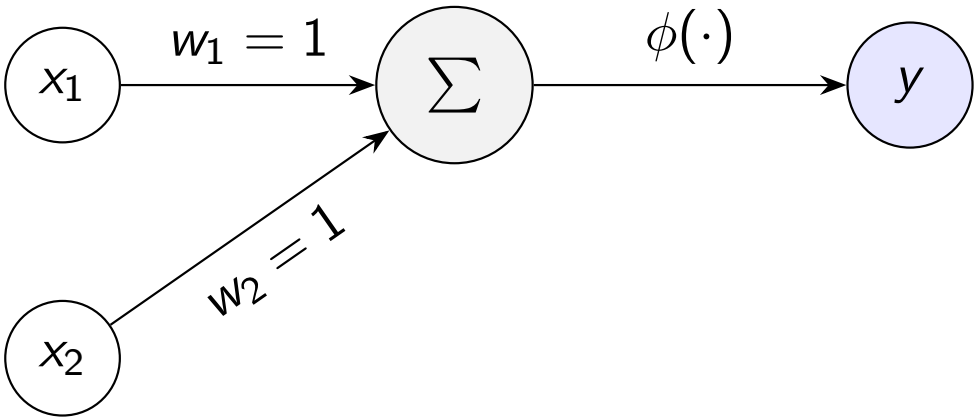
Alternatively, the perceptron can be written with a **threshold**:

$$y = \begin{cases} 1 & \text{if } \mathbf{w} \cdot \mathbf{x} \geq \theta \\ 0 & \text{otherwise} \end{cases}$$

Relationship: $b = -\theta$

Graphical Neuron: AND vs OR (Same $\theta = 2$)

AND



$$\theta = 2$$

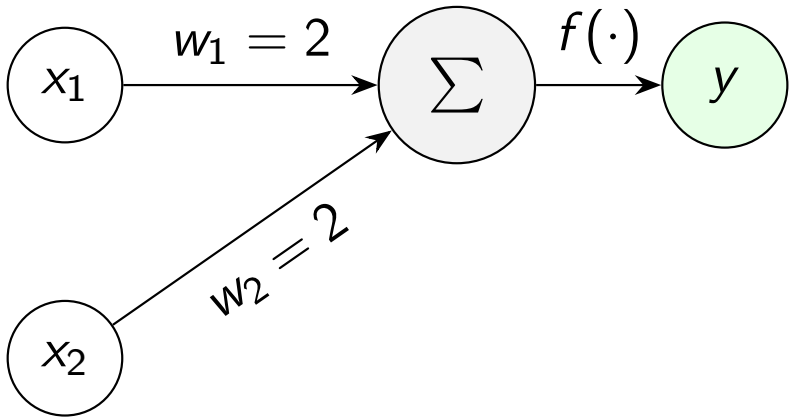
$$\text{So, } b = -2$$

$$\sum_w = w_1 x_1 + w_2 x_2 + b$$

$$= x_1 + x_2 + (-2)$$

$$\text{Output: } y = \phi(x_1 + x_2 - 2)$$

OR



$$\theta = 2$$

$$\text{So, } b = -2$$

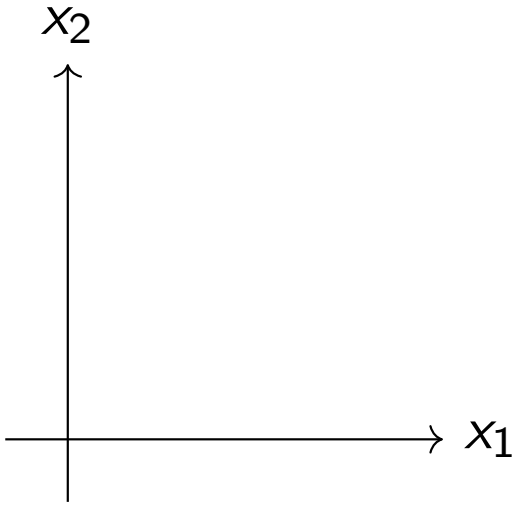
$$\sum_w = w_1 x_1 + w_2 x_2 + b$$

$$= 2x_1 + 2x_2 + (-2)$$

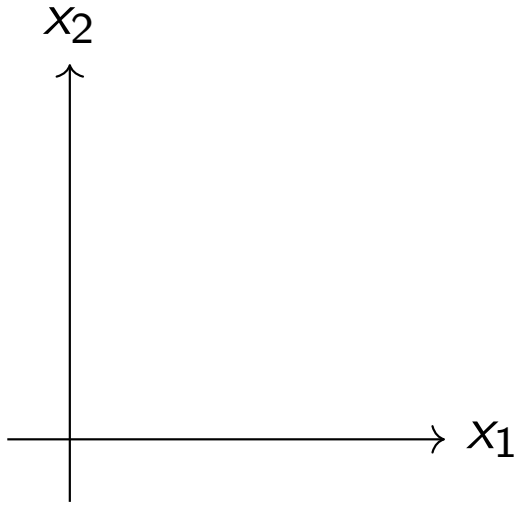
$$\text{Output: } y = \phi(2x_1 + 2x_2 - 2)$$

AND and OR Gate Feature Maps with Hyperplane Separation

AND Gate

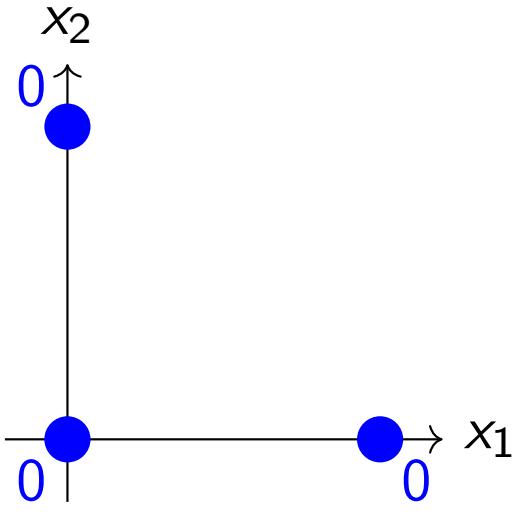


OR Gate

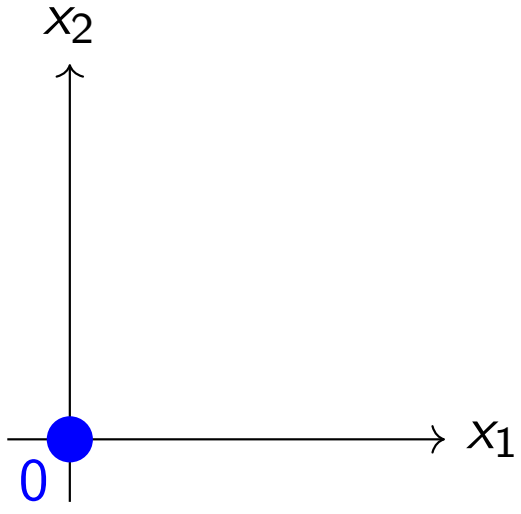


AND and OR Gate Feature Maps with Hyperplane Separation

AND Gate

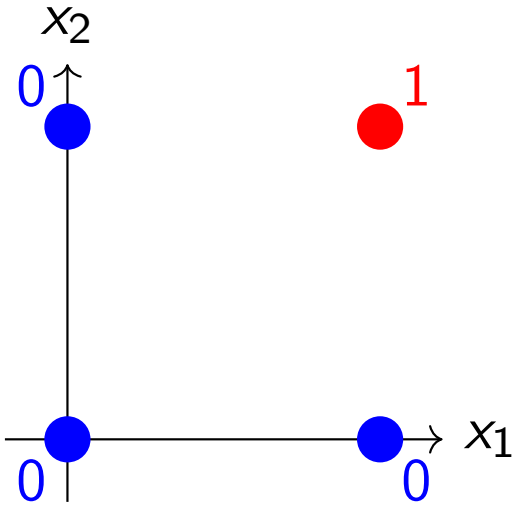


OR Gate

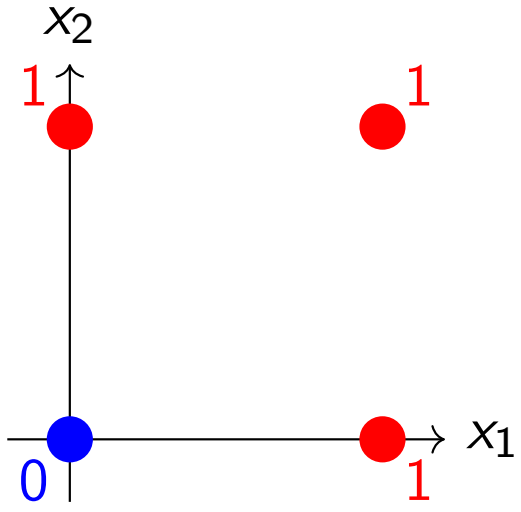


AND and OR Gate Feature Maps with Hyperplane Separation

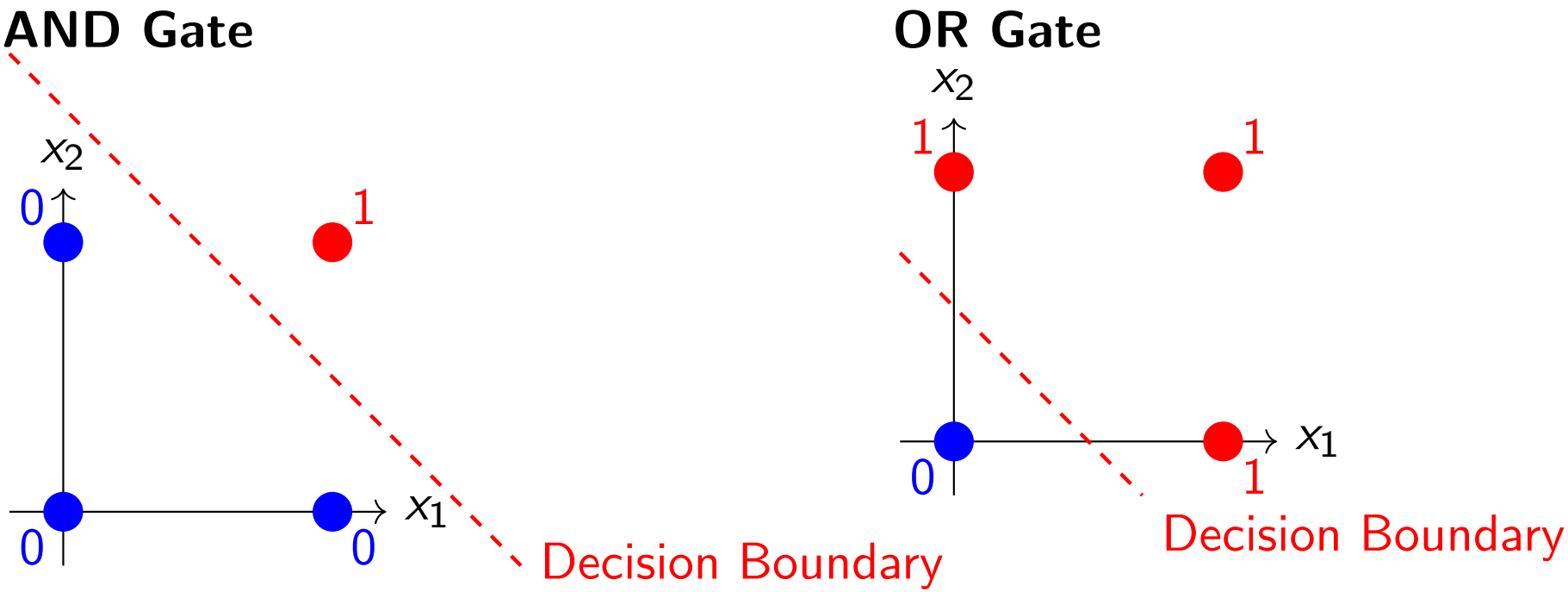
AND Gate



OR Gate



AND and OR Gate Feature Maps with Hyperplane Separation



Derivation of Perceptron Matrix Formula

1. Single perceptron (scalar form): For input vector $\mathbf{x} \in \mathbb{R}^n$, the output is:

$$y = f\left(\sum_{i=1}^{n} w_i x_i + b\right)$$

where w_i are weights, b is bias, and $f(\cdot)$ is the step function.

Derivation of Perceptron Matrix Formula

1. **Single perceptron (scalar form):** For input vector $\mathbf{x} \in \mathbb{R}^n$, the output is:

$$y = f\left(\sum_{i=1}^{n} w_i x_i + b\right)$$

where w_i are weights, b is bias, and $f(\cdot)$ is the step function.

2. **Vector form:** Using linear algebra, the same formula becomes:

$$y = f(\mathbf{w}^T \mathbf{x} + b), \quad \mathbf{w} = \begin{bmatrix} w_1 \\ w_2 \\ \vdots \\ w_n \end{bmatrix}, \ \mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}$$

Derivation of Perceptron Matrix Formula

1. **Single perceptron (scalar form):** For input vector $\mathbf{x} \in \mathbb{R}^n$, the output is:

$$y = f\left(\sum_{i=1}^{n} w_i x_i + b\right)$$

where w_i are weights, b is bias, and $f(\cdot)$ is the step function.

2. **Vector form:** Using linear algebra, the same formula becomes:

$$y = f(\mathbf{w}^T \mathbf{x} + b), \quad \mathbf{w} = \begin{bmatrix} w_1 \\ w_2 \\ \vdots \\ w_n \end{bmatrix}, \ \mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}$$

3. **Multiple input samples:** Let there be m input samples $\mathbf{x}^{(1)}, \ldots, \mathbf{x}^{(m)}$:

$$\mathbf{X} = \begin{bmatrix} | & | & & | \\ \mathbf{x}^{(1)} & \mathbf{x}^{(2)} & \ldots & \mathbf{x}^{(m)} \\ | & | & & | \end{bmatrix} \in \mathbb{R}^{n \times m}$$

Each column is a sample.

Derivation of Perceptron Matrix Formula

4. Linear combination for all samples:

$$\mathbf{Z} = \mathbf{W}^T \mathbf{X} \in \mathbb{R}^{1 \times m}$$

- $\mathbf{W}^T$ is $1 \times n$ - $\mathbf{X}$ is $n \times m$ - $\mathbf{Z}$ contains the weighted sum for each sample

Derivation of Perceptron Matrix Formula

4. Linear combination for all samples:

$$\mathbf{Z} = \mathbf{W}^T \mathbf{X} \in \mathbb{R}^{1 \times m}$$

- $\mathbf{W}^T$ is $1 \times n$ - $\mathbf{X}$ is $n \times m$ - $\mathbf{Z}$ contains the weighted sum for each sample

5. Adding bias: Bias must be added for each sample:

$$\mathbf{B} = \begin{bmatrix} b & b & \dots & b \end{bmatrix} \in \mathbb{R}^{1 \times m}$$

$$\mathbf{Z} + \mathbf{B} = \mathbf{W}^T \mathbf{X} + \mathbf{B}$$

Derivation of Perceptron Matrix Formula

4. Linear combination for all samples:

$$\mathbf{Z} = \mathbf{W}^T \mathbf{X} \in \mathbb{R}^{1 \times m}$$

- $\mathbf{W}^T$ is $1 \times n$ - $\mathbf{X}$ is $n \times m$ - $\mathbf{Z}$ contains the weighted sum for each sample

5. Adding bias: Bias must be added for each sample:

$$\mathbf{B} = \begin{bmatrix} b & b & \dots & b \end{bmatrix} \in \mathbb{R}^{1 \times m}$$

$$\mathbf{Z} + \mathbf{B} = \mathbf{W}^T \mathbf{X} + \mathbf{B}$$

6. Applying activation function: Finally, the perceptron output vector is:

$$\mathbf{y} = f(\mathbf{W}^T \mathbf{X} + \mathbf{B}) \in \mathbb{R}^{1 \times m}$$

Each element corresponds to the output for a sample.